## Solutions

**Q1.** What happens to the number of model parameters and the classification accuracy when switching from the MLP model to the CNN model? How can you explain this behavior?

**Answer.** The classification accuracy remains approximately the same (around 96%), even though the number of parameters changes significantly. The MLP model contains a large number of parameters (397,510) because the input image is first flattened into a vector and then processed by fully connected layers. The CNN model, instead, uses convolutional layers, which share weights across filters, thus using fewer parameters (32,842).

---

**Q2.** Why is it useful to keep the model small and efficient before applying quantization and deployment on edge hardware?

**Answer.** Smaller models typically require less memory, fewer computations, and lower power consumption. These properties are particularly important for edge devices, such as embedded systems or FPGAs, where hardware resources are limited.

---

**Q3.** Try implementing a lightweight CNN model (`CNN_lightweight`) in which the number of convolutional filters is reduced by half compared to the original CNN architecture. How would you modify the model definition to implement this change? After training the model for 5 epochs, how do the number of parameters and the test accuracy change compared to the original CNN?

**Answer.** Example of implementation.

Add the following model in the `./src/models.py` file:

```
import tensorflow as tf
from tensorflow.keras.layers import Conv2D, BatchNormalization, ReLU,
GlobalAveragePooling2D, Dense
from tensorflow.keras.models import Model

def CNN_lightweight(input_shape, N_classes):
    """
    Lightweight CNN: same architecture as custom_CNN_v1 but with half the
filters.
    custom_CNN_v1: 16 -> 32 -> 64
    CNN_lightweight: 8 -> 16 -> 32
    """
    img_input = tf.keras.Input(shape=input_shape)

    x = Conv2D(8, 5, strides=2, padding="same")(img_input)
    x = BatchNormalization()(x); x = ReLU()(x)

    x = Conv2D(16, 5, strides=2, padding="same")(x)
    x = BatchNormalization()(x); x = ReLU()(x)

    x = Conv2D(32, 3, strides=2, padding="same")(x)
    x = BatchNormalization()(x); x = ReLU()(x)

    x = GlobalAveragePooling2D()(x)
    outputs = Dense(N_classes, activation="softmax")(x)

    return Model(img_input, outputs, name="CNN_lightweight")
```

Then, in the notebook, import and instantiate the model:

```
from src.models import CNN_lightweight
model = CNN_lightweight(
    input_shape=x_train.shape[1:],
    N_classes=N_classes
)
```

You can print the number of trainable parameters using:

```
model.summary()
```

Train the model for 5 epochs and evaluate it on the test dataset following the same procedure used in this notebook.

The lightweight model has 8,618 parameters (compared to 32,842 in the original CNN) and achieves a slightly lower test accuracy (around 94%). This illustrates an important concept for edge AI: we can drastically reduce the model size (and therefore memory and compute

requirements) while keeping the accuracy reasonably high, especially on simple datasets like MNIST.

---

**Q4.** How does the model accuracy change compared to the model disk size after quantization?

**Answer.** For the MLP model, the test accuracy remains approximately 96% while the model size decreases significantly:

- FP32 file size: **4.57 MiB**
- INT8 file size: **1.55 MiB**
- Size reduction: **66%**

Although a small drop in accuracy is often expected after quantization, for simple models and datasets such as MNIST, the impact on accuracy can be very limited. This shows that quantization can greatly reduce the memory footprint of the model while maintaining almost the same performance.

---

**Q5.** Try changing the quantization strategy in: `quantizer = vitis_quantize.VitisQuantizer(model, quantize_strategy="pof2s")`. For example, replace `pof2s` with `fs`. Then repeat PTQ and try to compile the quantized model using the Vitis AI compiler. Do you notice anything unusual?

**Answer.** When using `fs` instead of `pof2s`, the quantized model may still be generated, but the compiler fails during the conversion to `.xmodel`. In this case, the compilation stops with the error `AssertionError: assert bias_pos is not None`. This happens because the DPU compiler expects a quantization format compatible with the target DPU, and `pof2s` (power-of-two scaling) is the only strategy supported by the DPU.